



Task 6: Improve the Model with Feature Scaling and Evaluation

Description:

Scale all features to the same range (0 to 1) and re-train the k-NN model. Compare performance with the unscaled model.

Explanation:

k-NN uses distance calculations. If one feature (e.g., petal length) has much larger numbers than others (e.g., sepal width), it dominates the distance. Scaling ensures every feature contributes equally. Common scaling is MinMaxScaler.

Example (Python code):

```
python

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import classification_report, confusion_matrix

# Create scaler and transform training & test data
scaler = MinMaxScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)    # use same scaling as training

# Train new k-NN on scaled data
knn_scaled = KNeighborsClassifier(n_neighbors=3)
knn_scaled.fit(X_train_scaled, y_train)

# Evaluate
y_pred_scaled = knn_scaled.predict(X_test_scaled)
new_accuracy = knn_scaled.score(X_test_scaled, y_test)
print(f"Scaled test accuracy: {new_accuracy * 100:.2f}%")

# Detailed report
print("\nClassification Report:")
print(classification_report(y_test, y_pred_scaled,
target_names=iris.target_names))

# Confusion matrix (how many were misclassified)
print("\nConfusion Matrix:")
```

```
print(confusion_matrix(y_test, y_pred_scaled))
```

What the student learns:

- Scaling often improves k-NN accuracy (sometimes from 95% to 97% or 100%).
- Classification report shows precision, recall, f1-score for each species.
- Confusion matrix reveals which species are confused (e.g., versicolor vs virginica).